

III Sales Performance Analytics: Ultimate Interview Study Guide (Session 1 & 2)

This comprehensive guide is the consolidated study manual for your upcoming interviews. It preserves all technical notes from **Phases 1 to 3** and adds today's discussions on **Phase 4 (Advanced SQL queries)**, **Phase 5 (Power BI Data Modeling & Dashboards)**, and a **comprehensive interview question bank with model answers**.

□ PHASE 1: Project Genesis & Business Value

1. Why Was This Project Built?

In corporate environments, transactional data is often stored in raw, denormalized Excel spreadsheets or flat CSV logs. While this format is convenient for simple viewing, it is highly inefficient for database storage, querying, and reporting.

We built this project to demonstrate a complete data engineering pipeline that cleans raw retail data (using Python), structures it into a high-performance **3NF Star-Schema relational database** (using MySQL), analyzes it (using SQL queries), and visualizes it (using Power BI).

2. The Problems of Flat Files (The Three Anomalies)

Before normalization, all sales, customer, product, and geographic columns were mixed in a single wide table called `superstore_raw`. This denormalized structure creates three severe operational risks:

1. Insertion Anomaly:

- *Definition:* You cannot insert a row for an entity unless you also insert unrelated data.
- *Superstore Example:* If you acquire a new customer (e.g., Jane Doe) but they have not placed an order yet, you **cannot** add their profile to the database. In a flat sheet, every row is a transaction. Since there is no transaction `order_id` or `product_id` for Jane, you cannot insert her name.

2. Update Anomaly:

- *Definition:* Changing a value in one place requires updating it in many rows, risking data inconsistency.
- *Superstore Example:* If a product name changes (e.g., from "Newell 322" to "Newell 322 Pack of 3"), you must scan and update thousands of historical records containing that `product_id`. If the query fails to update even a single row, the database is left in an inconsistent state where the same ID points to different names.

3. Deletion Anomaly:

- *Definition:* Deleting a record to remove one piece of information accidentally deletes entirely unrelated data.

- *Superstore Example:* If a customer named "Claire Gute" has placed only one order in our store's history, and we delete that order (due to a return or cancellation), we permanently lose her customer ID, name, segment, and geographical records.

3. Storage Redundancy (Storage Efficiency Math)

In a flat sheet of **10,000 rows**:

- A customer's name (e.g., "Brosina Hoffman" = 15 characters = 15 bytes in ASCII) is repeated on every line they buy. If they buy 50 items, that's $15 \times 50 = 750$ bytes for just their name.
 - By normalizing this into a `customers` table, we store the name "Brosina Hoffman" **exactly once** (15 bytes). In the transaction table, we store only their `customer_id` (e.g., "BH-11710" = 8 bytes) or an integer key (4 bytes).
 - **The Math:** For 10,000 transactions across 800 unique customers:
 - *Denormalized:* $10,000 \text{ rows} \times 15 \text{ bytes} = 150 \text{ KB}$ (just for customer names).
 - *Normalized:* $(800 \text{ unique customers} \times 15 \text{ bytes}) + (10,000 \text{ transactions} \times 4 \text{ byte integer keys}) = 12 \text{ KB} + 40 \text{ KB} = 52 \text{ KB}$.
 - *Result:* Over **65% storage reduction** on a single field! When scaled to millions of rows and dozens of columns, this saves gigabytes of memory and allows database pages to fit entirely into RAM.
-

□ PHASE 2: Database Schema & Normalization (1NF → 3NF)

1. Fact vs. Dimension Tables (The "Nouns" vs. "Verbs")

We split the raw dataset by separating the **descriptive context (Dimensions)** from the **quantitative measurements (Facts)**:

- **Dimension Tables (Contextual Data - The "Nouns"):**
 - `customers`: Customer profiles (`customer_id` (PK), `customer_name`, and `segment`).
 - `products`: Product catalog attributes (`product_id` (PK), `product_name`, `category`, and `sub_category`).
 - `regions`: Geographic registry (`region_id` (PK), `country`, `region`, `state`, `city`, and `postal_code`).
 - `orders`: Order header metadata (`order_id` (PK), `order_date`, `ship_date`, `ship_mode`, `customer_id` (FK), and `region_id` (FK)).
- **Fact Table (The Measurements - "Verbs"):**
 - `sales`: The transaction line-item records (`order_id` (FK), `product_id` (FK) -

combined as a composite Primary Key, sales revenue, quantity, discount, and profit).

2. Cardinality & Table Relationships

Our Star Schema implements a series of **One-to-Many (1:N)** relationships pointing from the Dimension tables to the Fact table:

- customers (1) $\rightarrow$ (N) orders
- regions (1) $\rightarrow$ (N) orders
- orders (1) $\rightarrow$ (N) sales (Fact)
- products (1) $\rightarrow$ (N) sales (Fact)
- *Note:* The **Many-to-Many (N:M)** relationship between orders and products is resolved using the sales table as a bridge table with a composite primary key (order_id, product_id).

3. Step-by-Step Normalization

We normalized the database from a single flat table into Third Normal Form (3NF):

- **First Normal Form (1NF):** Enforced atomic values in columns (no lists/arrays in cells) and defined unique rows using primary keys.
- **Second Normal Form (2NF):** Eliminated **Partial Dependency** (where non-key columns depend on only part of a composite key). Customer and product details were moved into their own tables so they depend purely on their single primary keys (customer_id and product_id).
- **Third Normal Form (3NF):** Eliminated **Transitive Dependency** (where non-key columns depend on other non-key columns). In orders, geographical fields depended on postal_code rather than directly on order_id. We extracted them into regions lookup records.

⚙️ PHASE 3: ETL Pipeline & Data Ingestion

Staging tables serve as entry zones for raw files, protecting the target production database from bad schemas or lock-ups.

1. Ingestion Sequences & SQL ETL Queries

To populate our schema, we executed these SQL scripts:

```
-- 1. Customers
INSERT INTO customers (customer_id, customer_name, segment)
SELECT customer_id, MIN(customer_name), MIN(segment)
FROM superstore_raw
GROUP BY customer_id;

-- 2. Products
INSERT INTO products (product_id, product_name, category, sub_category)
SELECT product_id, MIN(product_name), MIN(category), MIN(sub_category)
```

```

FROM superstore_raw
GROUP BY product_id;

-- 3. Regions
INSERT INTO regions (country, region, state, city, postal_code)
SELECT DISTINCT country, region, state, city, postal_code
FROM superstore_raw;

-- 4. Orders
INSERT INTO orders (order_id, order_date, ship_date, ship_mode, customer_id,
region_id)
SELECT DISTINCT
    s.order_id,
    STR_TO_DATE(s.order_date, '%Y-%m-%d'),
    STR_TO_DATE(s.ship_date, '%Y-%m-%d'),
    s.ship_mode,
    s.customer_id,
    r.region_id
FROM superstore_raw s
JOIN regions r
    ON s.country = r.country AND s.region = r.region
    AND s.state = r.state AND s.city = r.city AND s.postal_code = r.postal_code;

-- 5. Sales
INSERT INTO sales (order_id, product_id, sales, quantity, discount, profit)
SELECT order_id, product_id, SUM(sales), SUM(quantity), AVG(discount),
SUM(profit)
FROM superstore_raw
GROUP BY order_id, product_id;

```

2. Bulk Loading Optimization (LOAD DATA INFILE)

- **What we did:** In `02_data_loading.sql`, we used MySQL's `LOAD DATA LOCAL INFILE` command to load `Superstore_Clean.csv` directly into `superstore_raw`.
- **Why it matters for interviews:** standard SQL `INSERT` statements parse rows one-by-one, which is slow. `LOAD DATA LOCAL INFILE` bypasses standard SQL parsing layers and streams raw text files straight into database pages. For large datasets, this runs **100x faster**, turning a 10-minute load into a 5-second stream.

3. Database Indexes (B-Tree Performance Tuning)

- MySQL automatically indexes columns defined as `PRIMARY KEY`.
- **The Interview explanation:** "When we query the database, SQL uses **B-Tree Indexes** on keys like `customer_id` and `product_id`. Instead of executing an $O(N)$ full table scan, the search runs in $O(\log N)$ logarithmic time. When we execute joins across `sales` $\rightarrow$ `orders` $\rightarrow$ `customers` on their primary/foreign keys, indexing ensures that queries run in milliseconds rather than seconds."

□ Three Key Technical Hurdles & Solutions

1. String Trimming (Silent Join Failures):

- *Challenge:* IDs in the raw CSV had hidden trailing whitespaces (e.g. `'CG-12520 '`). This caused standard database joins to fail silently.

- *Solution:* Preprocessed text fields in Python using `.str.strip()` to clean all strings prior to importing.

2. Product Name Spelling Duplicates:

- *Challenge:* The same `product_id` had slight spelling differences across transaction lines (e.g., typos, formatting changes). Selecting distinct rows yielded key violations.
- *Solution:* Grouped by `product_id` and selected the alphabetical minimum `MIN(product_name)` to force key uniqueness.

3. Composite Key Violations in sales:

- *Challenge:* The same product was sometimes purchased multiple times in a single order, creating duplicate `(order_id, product_id)` entries that violated the primary key constraints.
- *Solution:* Rolled up duplicate rows by grouping by `order_id` and `product_id` and using aggregations (`SUM(sales)`, `SUM(quantity)`, `AVG(discount)`, `SUM(profit)`).

🔍 DATABASE SCHEMA CATALOG & RELATIONSHIPS

This section lists the structural mapping of all tables, detailing columns, key types, unique constraints, and policies for duplicate values.

1. Table Columns & Key Definitions

Table: customers

- **Purpose:** Stores unique profile records for retail buyers.
- **Columns:**
 - `customer_id` (VARCHAR): **Primary Key**. Must be completely unique. No duplicates allowed.
 - `customer_name` (VARCHAR): Non-key attribute. Duplicates allowed (multiple customers can share the same name).
 - `segment` (VARCHAR): Non-key attribute. Duplicates allowed (Consumer, Corporate, etc. repeat across rows).

Table: products

- **Purpose:** Stores unique metadata for retail catalog items.
- **Columns:**
 - `product_id` (VARCHAR): **Primary Key**. Must be completely unique. No duplicates allowed.
 - `product_name` (VARCHAR): Non-key attribute. Duplicates allowed (different

versions or identical product titles).

- **category** (VARCHAR): Non-key attribute. Duplicates allowed (Furniture, Office Supplies repeat).
- **sub_category** (VARCHAR): Non-key attribute. Duplicates allowed (Phones, Binders repeat).

Table: regions

- **Purpose:** Stores distinct geographical addresses to resolve shipping dependencies.
- **Columns:**
 - **region_id** (INT): **Primary Key (Auto-Incremented)**. Must be completely unique. No duplicates allowed.
 - **country** (VARCHAR): Non-key. Duplicates allowed (e.g., "United States" repeats).
 - **region** (VARCHAR): Non-key. Duplicates allowed (e.g., "West", "East").
 - **state** (VARCHAR): Non-key. Duplicates allowed (e.g., "California", "Kentucky").
 - **city** (VARCHAR): Non-key. Duplicates allowed (e.g., "Los Angeles", "Henderson").
 - **postal_code** (VARCHAR): Non-key. Duplicates allowed (multiple transactions map to same postal code).

Table: orders

- **Purpose:** Groups transaction items under a single purchase invoice context.
- **Columns:**
 - **order_id** (VARCHAR): **Primary Key**. Must be completely unique. No duplicates allowed.
 - **order_date** (DATE): Non-key. Duplicates allowed (multiple orders placed on the same day).
 - **ship_date** (DATE): Non-key. Duplicates allowed.
 - **ship_mode** (VARCHAR): Non-key. Duplicates allowed (Standard Class, First Class, etc.).
 - **customer_id** (VARCHAR): **Foreign Key** pointing to `customers(customer_id)`. Duplicates allowed (a customer can place multiple orders).
 - **region_id** (INT): **Foreign Key** pointing to `regions(region_id)`. Duplicates allowed (multiple orders shipped to same region).

Table: sales (Fact Table)

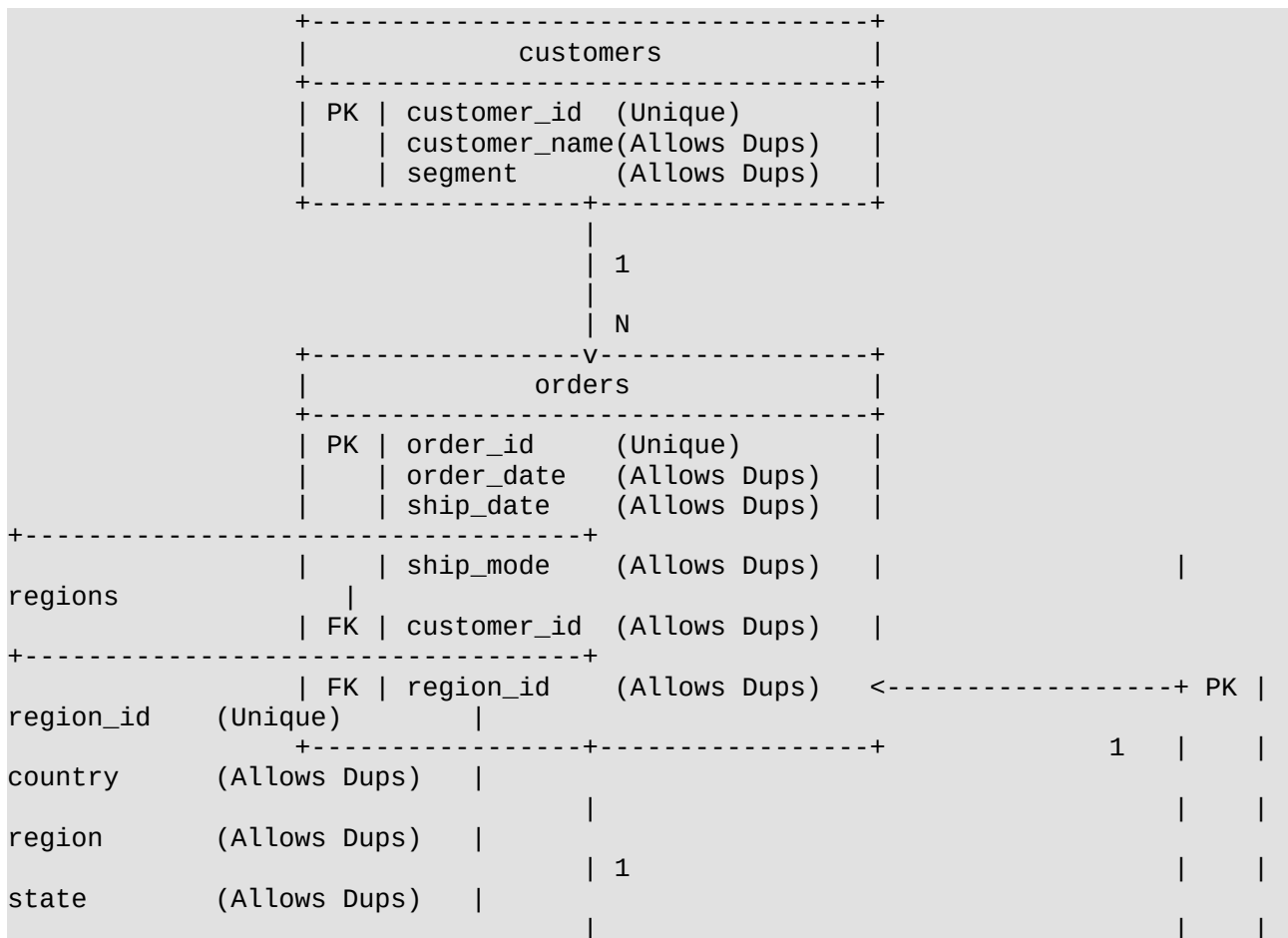
- **Purpose:** Stores transaction line-item quantities and monetary measurements.

- **Columns:**

- `order_id` (VARCHAR): **Composite Primary Key & Foreign Key** pointing to `orders(order_id)`. Duplicates allowed on its own (an order has multiple items).
- `product_id` (VARCHAR): **Composite Primary Key & Foreign Key** pointing to `products(product_id)`. Duplicates allowed on its own (a product is sold across multiple orders).
- `sales` (DECIMAL): Non-key fact. Duplicates allowed.
- `quantity` (INT): Non-key fact. Duplicates allowed.
- `discount` (DECIMAL): Non-key fact. Duplicates allowed.
- `profit` (DECIMAL): Non-key fact. Duplicates allowed.
- *Composite Key Rule*: The combined pair (`order_id`, `product_id`) must be completely unique. No duplicate rows can contain the exact same order and product ID combination.

2. Relationship Keys Map

This diagram illustrates how primary keys (PK) connect to foreign keys (FK) across the database schema, showing the cardinality relationships:



city	(Allows Dups)				N			
postal_code	(Allows Dups)							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
			sales					
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
			PK		order_id	(Allows Dups)*		
			PK		product_id	(Allows Dups)*		
					sales	(Allows Dups)		
					quantity	(Allows Dups)		
					discount	(Allows Dups)		
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
					profit	(Allows Dups)		
products					*Unique in Composite PK			
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
								PK
product_id	(Unique)							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
product_name	(Allows Dups)							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
category	(Allows Dups)							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
sub_category	(Allows Dups)							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
								1
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								

PHASE 4: SQL Business Analytics (Queries 6 to 30)

Queries 1 to 5: High-Level business KPIs

- **Query 1: Overall business performance**

```
SELECT ROUND(SUM(sales), 2) AS total_revenue, SUM(quantity) AS
total_units_sold, ROUND(SUM(profit), 2) AS total_profit FROM sales;
```

- **Query 2: Database scale analysis (Independent subqueries)**

```
SELECT (SELECT COUNT(*) FROM customers) AS total_customers, (SELECT
COUNT(*) FROM products) AS total_products, (SELECT COUNT(*) FROM orders)
AS total_orders;
```

- **Query 3 & 4: AOV & Average profit per order**

```
SELECT ROUND(SUM(sales) / COUNT(DISTINCT order_id), 2) AS
average_order_value FROM sales;
```

- **Query 5: Profit margin percentage**

```
SELECT ROUND((SUM(profit) / SUM(sales)) * 100, 2) AS
profit_margin_percentage FROM sales;
```

Queries 6 to 10: Category & product rankings

- **Query 6 & 7: Sales & profit by category/sub-category**


```
SELECT p.category, ROUND(SUM(s.sales), 2) AS total_sales,  
ROUND(SUM(s.profit), 2) AS total_profit, SUM(s.quantity) AS units_sold  
FROM sales s JOIN products p ON s.product_id = p.product_id GROUP BY  
p.category ORDER BY total_sales DESC;
```

- **Query 8, 9 & 10: Top 10 products by revenue/profit/quantity**

```
SELECT p.product_name, ROUND(SUM(s.sales), 2) AS total_revenue  
FROM sales s JOIN products p ON s.product_id = p.product_id  
GROUP BY p.product_id, p.product_name ORDER BY total_revenue DESC LIMIT  
10;
```

Queries 11 to 15: Anomaly checking & customer analysis

- **Query 11: Identifying loss-making products (HAVING filters on aggregate calculations)**

```
SELECT p.product_name, ROUND(SUM(s.profit), 2) AS total_loss  
FROM sales s JOIN products p ON s.product_id = p.product_id  
GROUP BY p.product_id, p.product_name HAVING SUM(s.profit) < 0 ORDER BY  
total_loss ASC LIMIT 10;
```

- **Query 12: Category contribution to total sales % (Nested subquery division)**

```
SELECT p.category, ROUND(SUM(s.sales), 2) AS total_sales,  
ROUND((SUM(s.sales) / (SELECT SUM(sales) FROM sales)) * 100, 2) AS  
sales_contribution_percent  
FROM sales s JOIN products p ON s.product_id = p.product_id GROUP BY  
p.category;
```

- **Query 13 & 14: Top 10 customers by revenue/profit (Three-table joins)**

```
SELECT c.customer_name, ROUND(SUM(s.sales), 2) AS total_revenue  
FROM sales s JOIN orders o ON s.order_id = o.order_id JOIN customers c ON  
o.customer_id = c.customer_id  
GROUP BY c.customer_id, c.customer_name ORDER BY total_revenue DESC LIMIT  
10;
```

- **Query 15: Customer segment analysis**

```
SELECT c.segment, ROUND(SUM(s.sales), 2) AS total_sales, COUNT(DISTINCT  
c.customer_id) AS total_customers  
FROM sales s JOIN orders o ON s.order_id = o.order_id JOIN customers c ON  
o.customer_id = c.customer_id GROUP BY c.segment;
```

Queries 16 to 20: Temporal trends & customer averages

- **Query 16: Average revenue per customer**

```
SELECT ROUND(SUM(s.sales) / COUNT(DISTINCT c.customer_id), 2) AS  
avg_revenue_per_customer  
FROM sales s JOIN orders o ON s.order_id = o.order_id JOIN customers c ON  
o.customer_id = c.customer_id;
```

- **Query 17: Top customers by number of orders**

```
SELECT c.customer_name, COUNT(o.order_id) as total_orders  
FROM orders o JOIN customers c ON o.customer_id = c.customer_id GROUP BY
```

```
c.customer_id, c.customer_name ORDER BY total_orders DESC LIMIT 10;
```

- **Query 18, 19 & 20: Yearly & monthly trends (YEAR() / MONTH() date extraction)**

```
SELECT YEAR(o.order_date) AS order_year, MONTH(o.order_date) AS  
order_month, ROUND(SUM(s.sales), 2) AS total_sales  
FROM orders o JOIN sales s ON o.order_id = s.order_id GROUP BY  
YEAR(o.order_date), MONTH(o.order_date) ORDER BY order_year, order_month;
```

Queries 21 to 25: Geography & pricing discounts

- **Query 21, 22 & 23: Regional, state & city breakdowns**

```
SELECT r.city, ROUND(SUM(s.sales), 2) AS total_sales, ROUND(SUM(s.profit),  
2) AS total_profit  
FROM regions r JOIN orders o ON o.region_id = r.region_id JOIN sales s ON  
o.order_id = s.order_id  
GROUP BY r.city ORDER BY total_sales DESC LIMIT 10;
```

- **Query 24: Discount impact on profit**

```
SELECT discount, ROUND(SUM(sales), 2) AS total_sales, ROUND(SUM(profit),  
2) AS total_profit FROM sales GROUP BY discount;
```

- **Query 25: High discount products**

```
SELECT p.product_name, ROUND(AVG(s.discount), 2) AS avg_discount,  
ROUND(SUM(s.profit), 2) AS total_profit  
FROM products p JOIN sales s ON p.product_id = s.product_id GROUP BY  
p.product_id, p.product_name HAVING AVG(s.discount) >= 0.30;
```

☐ Queries 26 to 30: Advanced window functions, CTEs & lag operations

Comparing Ranking Functions

An interviewer will ask you to explain this difference visually:

Function	Values	Result Ranks	Behavior
ROW_NUMBER()	Apple, Apple, Banana	1, 2, 3	Strictly sequential. No duplicates.
RANK()	Apple, Apple, Banana	1, 1, 3	Ties share rank. Skips next numbers.
DENSE_RANK()	Apple, Apple, Banana	1, 1, 2	Ties share rank. Does not skip numbers.

- **Query 26: Running total sales (SUM() OVER)**

```
SELECT YEAR(o.order_date) AS order_year, MONTH(o.order_date) AS  
order_month, ROUND(SUM(s.sales), 2) AS monthly_sales,  
ROUND(SUM(SUM(s.sales)) OVER (ORDER BY YEAR(o.order_date),  
MONTH(o.order_date)), 2) AS running_total_sales  
FROM orders o JOIN sales s ON o.order_id = s.order_id GROUP BY
```

```
YEAR(o.order_date), MONTH(o.order_date);
```

- **Query 27: Rank products by sales (RANK() OVER)**

```
SELECT p.product_name, ROUND(SUM(s.sales)) as total_sales,  
       RANK() OVER(ORDER BY SUM(s.sales) DESC) AS sales_rank  
FROM products p JOIN sales s ON p.product_id = s.product_id GROUP BY  
p.product_id, p.product_name;
```

- **Query 28: Top product in each category (Window Partition + CTE)**

```
WITH product_sales AS (  
    SELECT p.category, p.product_name, ROUND(SUM(s.sales), 2) AS  
    total_sales,  
           DENSE_RANK() OVER (PARTITION BY p.category ORDER BY  
SUM(s.sales) DESC) AS sales_rank  
    FROM products p JOIN sales s ON p.product_id = s.product_id GROUP BY  
p.category, p.product_id, p.product_name  
)  
SELECT category, product_name, total_sales FROM product_sales WHERE  
sales_rank = 1;
```

- **Query 29: Month-over-Month sales growth (LAG() OVER)**

```
WITH monthly_sales AS (  
    SELECT YEAR(o.order_date) AS order_year, MONTH(o.order_date) AS  
    order_month, ROUND(SUM(s.sales), 2) AS total_sales  
    FROM orders o JOIN sales s ON o.order_id = s.order_id GROUP BY  
YEAR(o.order_date), MONTH(o.order_date)  
)  
SELECT order_year, order_month, total_sales,  
       LAG(total_sales) OVER (ORDER BY order_year, order_month) AS  
previous_month_sales,  
       ROUND(total_sales - LAG(total_sales) OVER (ORDER BY order_year,  
order_month), 2) AS sales_difference  
FROM monthly_sales;
```

- **Query 30: Top 10 highest revenue orders**

```
SELECT o.order_id, c.customer_name, ROUND(SUM(s.sales)) as  
total_order_value  
FROM orders o JOIN customers c ON o.customer_id = c.customer_id JOIN sales  
s ON o.order_id = s.order_id  
GROUP BY o.order_id, c.customer_name ORDER BY total_order_value DESC LIMIT  
10;
```

III PHASE 5: Power BI Data Modeling & Dashboards

1. Data Model Relationships

All lookup dimension tables flow down to the central fact table in a one-directional relationship:

- customers (1) $\rightarrow$ (N) orders
- regions (1) $\rightarrow$ (N) orders
- orders (1) $\rightarrow$ (N) sales (Fact)

- products (1) $\rightarrow$ (N) sales (Fact)
- *Filtering flow*: Set strictly to **Single (one-directional)** to avoid circular dependency loops and maximize model recalculation speeds.

2. Custom DAX Formulas

- Total Sales = SUM(sales[sales])
- Total Profit = SUM(sales[profit])
- Total Orders = DISTINCTCOUNT(sales[order_id])
- Profit Margin % = DIVIDE([Total Profit], [Total Sales], 0) * 100 (*Using DIVIDE handles division-by-zero errors safely*)
- Average Order Value (AOV) = DIVIDE([Total Sales], [Total Orders], 0)

3. Power BI Storage: Import Mode vs. DirectQuery

- **What we did**: We loaded data into Power BI using **Import Mode**.
- **Why we chose it (Interview Explanation)**:
 - *Import Mode*: Power BI takes a copy of the data and loads it into the local computer's RAM. It uses the **VertiPaq column-store compression engine** to reduce file sizes (by up to 10x) and index columns. This ensures that calculations and dashboard interactions respond instantly.
 - *DirectQuery*: This makes live queries directly to the MySQL database on every click, which is much slower and strains MySQL performance. Import Mode was optimal for our 10,000-row superstore catalog.

4. Actionable Business Insights Derived

1. **The Furniture Margin Problem**: Furniture top-line sales match Technology's volume (~\$742k vs \$836k), ~~but generated almost zero net profit (\$18k vs ~\$145k)~~ due to high shipping costs and aggressive discounting.
 2. **Geographic Focus**: The West (37%) and East (29%) regions drive over 65% of the company's revenue, making California and New York the primary operational focus states.
 3. **Segment Concentration**: Retail consumers represent over 50% of the customer transaction volume. B2B corporate segments represent expansion opportunities.
 4. **Logistics Core**: Standard Class shipping handles 60% of deliveries, suggesting bulk shipping rate renegotiations should focus exclusively on standard freight.
-

🎙 INTERVIEW QUESTION BANK & MODEL ANSWERS

Q1: Why did you choose a Star Schema over a Snowflake Schema or a single wide table?

- **The Answer:** "We selected the Star Schema for three reasons:
 1. **Fewer Joins:** It keeps our dimension tables denormalized, meaning analytical queries require fewer joins and run significantly faster compared to a Snowflake schema.
 2. **Dashboard Performance:** Power BI is natively optimized for Star Schemas because the filter direction flows unidirectionally from lookups directly to facts.
 3. **Anomalies:** A single wide table contains massive redundancy and suffers from insertion, update, and deletion anomalies. The Star Schema eliminates these anomalies while retaining high query efficiency."

Q2: Tell me about a time you had dirty data or spelling conflicts. How did you resolve them?

- **The Answer:** "During product extraction in our ETL pipeline, I discovered that the same `product_id` was associated with multiple product name spellings over time due to typos. Because `product_id` was our primary key in the products dimension, attempting a simple `SELECT DISTINCT` threw key violation errors.
- **Solution:** I grouped the records by `product_id` and selected the alphabetical minimum spelling using `MIN(product_name)`. This guaranteed exactly one clean product name per unique ID. Additionally, trailing whitespaces in IDs were cleaned programmatically in Python using `.str.strip()` to prevent silent join failures."

Q3: Why did you use `STR_TO_DATE` in MySQL instead of importing date columns directly?

- **The Answer:** "In the raw CSV dataset, transaction dates were stored in text string formats. In relational databases, storing dates as strings prevents the engine from indexing them chronologically. To enforce data integrity and enable fast temporal indexing, I imported them as `VARCHAR` to a staging table, and then migrated them to the target table using `STR_TO_DATE(order_date, '%Y-%m-%d')` to cast them into actual SQL `DATE` types."

Q4: When would you use a CTE instead of a subquery?

- **The Answer:** "Common Table Expressions (CTEs) are preferred over nested subqueries for two reasons:
 1. **Readability:** CTEs allow you to define a named temporary result set at the top of the query, making the code much easier to read and maintain than nested parenthetical subqueries.
 2. **Recursion:** CTEs support recursion (using `WITH RECURSIVE`), which is necessary for hierarchical data structures like organization charts or bill of materials."

Q5: Why can't you put a window function directly in the WHERE clause?

- **The Answer:** "SQL follows a strict evaluation order: $\text{FROM} \rightarrow \text{JOIN} \rightarrow \text{WHERE} \rightarrow \text{GROUP BY} \rightarrow \text{HAVING} \rightarrow \text{SELECT} \rightarrow \text{WINDOW} \rightarrow \text{ORDER BY}$ Because the WHERE clause is evaluated before window functions are calculated, the database doesn't know the window results yet at that stage of execution. To filter on a window rank, you must wrap the query in a CTE or a subquery and apply the filter in the outer query."

Q6: What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

- **The Answer:** "All three are ranking functions, but they handle ties differently:
 - ROW_NUMBER() assigns a unique sequential number to each row (e.g. 1, 2, 3, 4) with no duplicates, regardless of ties.
 - RANK() assigns the same rank to identical values but skips subsequent numbers (e.g., 1, 2, 2, 4).
 - DENSE_RANK() assigns the same rank to identical values without skipping numbers (e.g., 1, 2, 2, 3)."

Q7: What is the difference between explicit and implicit measures in Power BI?

- **The Answer:** "An **implicit measure** is created automatically when you drag a numerical column (like sales) directly onto a visual card. An **explicit measure** is written manually using DAX (e.g., `Total Sales = SUM(sales[sales])`). Explicit measures are the industry best practice because they are reusable across multiple reports, easy to maintain, and can handle complex logic like context filtering or division-by-zero errors."

Q8: Why use DIVIDE instead of the standard division operator / in DAX?

- **The Answer:** "The standard division operator / will fail or return NaN if the denominator evaluates to zero, which can crash visuals on a dashboard when filters are applied. The DIVIDE function automatically handles division-by-zero errors by returning a safe alternative value (like 0 or BLANK), keeping dashboards clean and error-free."

Q9: If a product category has high sales but negative profit, what actions would you recommend?

- **The Answer:** "This is exactly what occurred with our Furniture category. Despite generating ~\$742k in revenue, its profits were thin due to heavy shipping costs and steep discounts. I would recommend:
 1. **Discount Boundaries:** Setting a maximum discount threshold (e.g. limiting discount rates to 15% instead of 45%).
 2. **Logistics Renegotiation:** Since Standard Class shipping handles 60% of our orders,

renegotiating freight rates with standard carriers.

3. **Pricing Adjustments:** Shifting bulk shipping fees onto the buyer for heavy items instead of absorbing shipping costs in the base product price."

Q10: What is the significance of the composite primary key in the sales table?

- **The Answer:** "The sales table acts as a bridge table between orders and products to resolve their many-to-many relationship. The composite primary key (`order_id`, `product_id`) ensures that each row represents a unique line item—meaning a specific product sold within a specific order. This enforces referential integrity and prevents duplicate product lines from being inserted under the same transaction ID."

Q11: How do you optimize joining large tables? (Query Tuning)

- **The Answer:** "To optimize joins on large tables, we use several strategies:
 1. **Primary/Foreign Keys:** Ensure all joins are written on indexed keys (PKs and FKs), utilizing fast B-Tree indexing.
 2. **Filter First:** Apply WHERE filters before joining when using subqueries, reducing the volume of data that needs to be joined.
 3. **Select Specific Columns:** Avoid `SELECT *`. Selecting only the required columns reduces memory footprint during query execution.
 4. **Analyze Execution Plans:** Use the EXPLAIN keyword in MySQL to analyze the execution path, verifying that the optimizer is performing 'Index scans' instead of slow 'Full table scans'."

Q12: How would you scale this database to millions of transactions daily?

- **The Answer:** "If the transaction volume scaled to millions of rows daily, I would implement three changes:
 1. **Cloud Data Warehousing:** Migrate the MySQL database to a cloud-based columnar warehouse like Snowflake or Google BigQuery to run massive parallel queries (MPP).
 2. **dbt (Data Build Tool):** Implement dbt to handle SQL transformations. dbt organizes models, handles data lineage, and runs automated tests directly in the cloud warehouse.
 3. **Workflow Orchestration:** Use Apache Airflow to orchestrate the pipeline, scheduling raw file extractions, warehouse loads, and dbt runs, with automated error alerts."